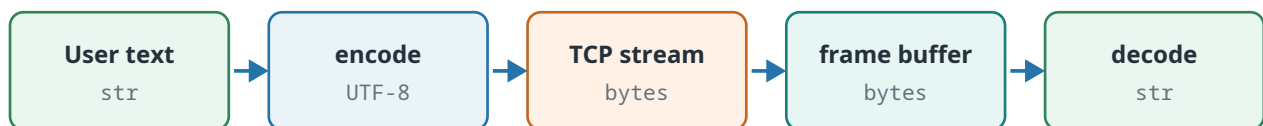


Python Bytes and TCP Message Framing

A step-by-step lesson for the threaded ephemeral chat server



Keep transport data as bytes; decode only complete frames.

Core outcome: receive arbitrary TCP chunks, preserve incomplete bytes, produce complete newline-delimited messages, and keep memory bounded.

Lesson 3 - TCP Chat Learning Project

1. What you will learn

This lesson connects Python's data types to the behavior of a real TCP connection. It focuses on the boundary between user text, transport bytes, and application messages.

- Distinguish `str`, `bytes`, and `bytearray`.
- Encode text before sending and decode only after a complete frame is available.
- Explain why `recv(1024)` returns up to 1,024 bytes, not one message.
- Reassemble split messages and separate combined messages.
- Give every client worker its own receive buffer.
- Enforce a maximum frame size and decide what EOF means for incomplete data.

Learning path

Step	Question	Evidence
1. Represent	What are bytes?	Inspect types, lengths, indexes, and encoded values.
2. Transport	What does TCP deliver?	Predict multiple valid <code>recv</code> chunkings.
3. Frame	Where does a message end?	Extract newline-delimited frames.
4. Bound	What can go wrong?	Reject oversized and malformed frames.
5. Test	Can it survive arbitrary chunks?	Pass deterministic unit tests without sockets.

Definition of done: your framing code returns the same complete messages regardless of how the same byte stream is divided across `recv` calls.

2. Python text and bytes

Python text is Unicode. A socket carries bytes. Encoding and decoding form the explicit boundary between those two representations.

```
text = "café" # str
wire = text.encode("utf-8") # bytes

print(type(text)) # <class 'str'>
print(wire) # b'caf\xc3\xa9'
print(len(text)) # 4 characters
print(len(wire)) # 5 bytes
print(wire.decode("utf-8")) # café
```

Indexing behaves differently

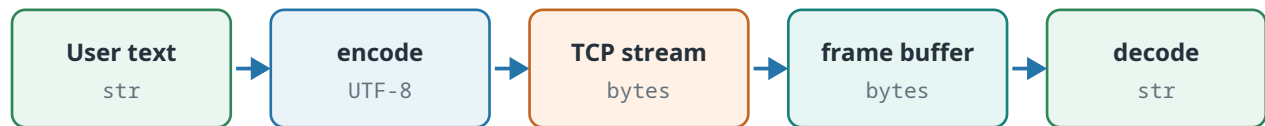
```
data = b"ABC"
data[0] # 65: indexing returns an integer
data[0:1] # b'A': slicing returns bytes
list(data) # [65, 66, 67]
```

Operations you will use

Expression	Result	Meaning
<code>b'\n' in data</code>	Boolean	A complete delimiter has arrived.
<code>left + right</code>	New bytes	Combine saved and newly received bytes.
<code>data.split(b'\n', 1)</code>	Two byte parts	Extract the first complete frame.
<code>data.decode('utf-8')</code>	Text	Interpret a complete byte frame as text.
<code>repr(data)</code>	Debug text	Make newlines and escaped bytes visible.

Do not mix the types accidentally: "hello" + b"world" raises `TypeError`.

3. The socket boundary



Keep transport data as bytes; decode only complete frames.

Sending

```
message_text = input("> ")
wire_frame = (message_text + "\n").encode("utf-8")
client.sendall(wire_frame)
```

The newline is part of the application protocol. `sendall()` hands the full encoded frame toward the operating system or raises an error. It does not confirm that another person read the message.

Receiving

```
chunk = connection.recv(BUFFER_SIZE)

if chunk == b'':
    # The peer closed its sending side.
    break
```

Value returned by <code>recv</code>	Interpretation
<code>b'hello\n'</code>	Some bytes are available. They may contain part, one, or several frames.
<code>b'h'</code>	Only one byte was available even if a larger <code>recv</code> size was requested.
<code>b''</code>	EOF: the peer closed its sending side. No more bytes will arrive.

An empty chat frame is `b'\n'`. A zero-byte result `b''` is connection state, not a message.

4. Why recv cannot define messages

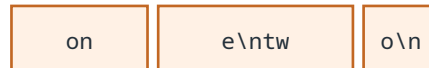
Application writes



Possible recv result A



Possible recv result B



TCP preserves byte order, not write or message boundaries.

The application may call `sendall()` once per message, but TCP exposes one ordered stream. The receiver chooses how much currently available data to return, capped by the `recv` argument.

Client action	Valid server observations
<code>sendall(b'hello\n')</code>	<code>b'hello\n'</code> or <code>b'he'</code> then <code>b'llo\n'</code>
Send <code>one\n</code> , then <code>two\n</code>	One combined chunk, two chunks, or several smaller chunks
Send 5,000 bytes; call <code>recv(1024)</code>	At least five positive receives are required, possibly more

Two separate limits

- **Chunk size:** how many bytes one `recv()` call may return. It is an implementation choice, not a protocol rule.
- **Frame size:** the largest message your protocol accepts. It is a protocol and resource-safety rule.

Changing `BUFFER_SIZE` changes chunking behavior and call frequency. It does not remove the need for framing.

5. The framing buffer, step by step

The buffer contains bytes received for one connection that have not yet been emitted as complete frames.

Event	Received chunk	Buffer after append	Frames emitted	Remainder
1	b'hel'	b'hel'	None	b'hel'
2	b'lo\nwor'	b'hello\nwor'	b'hello'	b'wor'
3	b'ld\nnext\n'	b'world\nnext\n'	b'world', b'next'	b''

Algorithm

```
buffer = b""

repeat:
    chunk = recv(up_to_BUFFER_SIZE_bytes)
    if chunk is EOF:
        apply the incomplete-frame policy and stop

    buffer = buffer + chunk

    while buffer contains b"\n":
        remove one complete byte frame
        decode that frame as UTF-8
        handle the resulting message

    enforce the maximum unfinished-frame size
```

Why the inner loop matters

A single chunk can contain b'one\ntwo\nthree\n'. Using `if` would process only one frame and leave two complete frames waiting until some future network activity. Using `while` drains every complete frame already available.

Invariant after processing: the remainder contains no newline and is the exact prefix of the next possible frame.

6. Unicode and decode timing

UTF-8 characters may contain more than one byte. TCP can divide the stream between those bytes, so decoding arbitrary recv chunks can reject valid text.

```
wire = "café\n".encode("utf-8")
# b'caf\xc3\xa9\n'

# A possible split:
chunk_1 = b"caf\xc3"
chunk_2 = b"\xa9\n"

chunk_1.decode("utf-8")
# UnicodeDecodeError: the final character is incomplete
```

Safe sequence

Stage	Representation	Action
Receive	Arbitrary bytes	Append chunks without decoding.
Frame	Complete byte frame	Remove the delimiter and enforce the byte limit.
Decode	Complete UTF-8 sequence	Use strict UTF-8; reject malformed frames.
Validate	Python text	Check empty text, commands, usernames, and content rules.

Protocol decisions

- Will an empty frame `b'\n'` be accepted or ignored?
- Can message text contain line breaks? If yes, newline framing needs escaping or replacement.
- What happens after invalid UTF-8: error response, dropped frame, or disconnected client?
- Is the maximum measured in encoded bytes or decoded characters? Use encoded bytes for transport safety.

For this chat: one terminal input line becomes one UTF-8 frame; embedded newlines are not allowed; malformed UTF-8 is a protocol error.

7. Requirements and failure policies

Requirement	Why it exists	Recommended policy
Delimiter	TCP supplies no message boundary.	Every frame ends in one <code>b'\n'</code> .
Maximum frame	A client can send forever without a delimiter.	Reject when unfinished data exceeds 4,096 bytes.
Encoding	Peers must interpret bytes consistently.	UTF-8 with strict decoding.
EOF policy	A client may close during a frame.	Discard the unfinished remainder and record metadata only.
Empty frame	<code>b'\n'</code> is a valid frame structurally.	Ignore or reject it explicitly.
Malformed UTF-8	Bad data must not crash a worker.	Send a protocol error or disconnect that client.
Ephemeral content	The product promises no history.	Never write message content to files or logs.

Important invariants

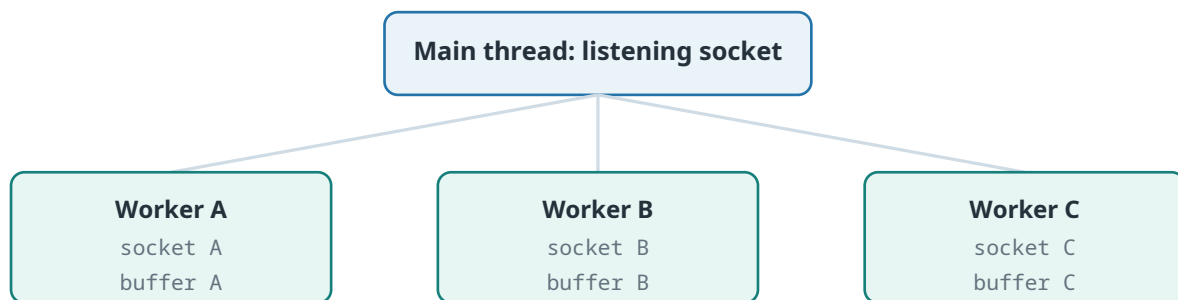
- Frames are emitted exactly once and in byte-stream order.
- After extraction, the remainder contains no complete delimiter.
- A worker never reads from another client's buffer or socket.
- Routing receives complete decoded messages, never raw chunks.
- No incomplete frame can grow beyond the configured limit.

Ephemeral does not mean zero memory

Bytes must exist transiently in kernel buffers and application memory while being received, framed, and relayed. The guarantee is that content is not persisted, retained as history, or included in logs. Keep temporary references short-lived and bounded.

A requirement becomes useful when it specifies observable behavior for success, malformed input, EOF, and resource exhaustion.

8. Ownership in the threaded server



Each connection has an independent ordered byte stream and independent incomplete state.

The main thread owns the listening socket. After `accept()`, one worker owns the connected socket and its incomplete-frame state. A framing helper may be shared as code, but its state must remain per connection.

A testable module boundary

```
# framing.py
def extract_frames(
    existing_buffer: bytes,
    received_chunk: bytes,
) -> tuple[list[bytes], bytes]:
    """Return complete frames and unfinished bytes."""
    ...

# server.py
from framing import extract_frames

buffer = b""
chunk = connection.recv(BUFFER_SIZE)
frames, buffer = extract_frames(buffer, chunk)
```

Why this separation is valuable

- Framing tests do not need ports, timing, threads, or live sockets.
- The server coordinates I/O; the framing module interprets byte structure.
- A future length-prefixed framer can replace the delimiter framer without rewriting socket ownership.

Start with a pure function so state movement is visible. A `LineFramer` class is appropriate later if it improves the interface rather than hiding the lesson.

9. Deterministic framing tests

The same logical stream should produce the same frames for every chunk division. These tests model TCP behavior without relying on nondeterministic network timing.

Existing	Incoming	Expected frames	Expected remainder
b''	b'hello\n'	[b'hello']	b''
b''	b'hel'	[]	b'hel'
b'hel'	b'lo\n'	[b'hello']	b''
b''	b'one\ntwo\n'	[b'one', b'two']	b''
b''	b'one\ntw'	[b'one']	b'tw'
b''	b'\n'	[b'']	b''

Property to reason about

```
stream = b"one\ntwo\npartial"

# These chunk sequences describe the same stream:
[stream]
[b"o", b"ne\ntwo\npartial"]
[b"one\n", b"two\n", b"partial"]

# Every sequence must finish with:
frames == [b"one", b"two"]
remainder == b"partial"
```

Add negative tests

- Unfinished data exceeds the maximum frame size.
- A frame is exactly at the size limit.
- A complete frame contains malformed UTF-8.
- EOF arrives while the remainder is non-empty.

Unit tests should control chunk boundaries deliberately. Integration tests should verify that the framing module is wired into the real worker correctly.

10. Hands-on lesson

Part A - Explore bytes in the Python shell

```
text = "café"
wire = text.encode("utf-8")
type(text), type(wire)
len(text), len(wire)
list(wire)
wire[0], wire[0:1]
wire.decode("utf-8")
```

Before pressing Enter, predict every result. Explain why character count and byte count differ.

Part B - Manually reconstruct a frame

```
buffer = b""
buffer += b"hel"
b"\n" in buffer
buffer += b"lo\nnext"
message, buffer = buffer.split(b"\n", 1)
message, buffer, message.decode("utf-8")
```

Expected final state: `message == b'hello'` and `buffer == b'next'`.

Part C - Implement the framing contract

- Create `framing.py` beside `server.py`.
- Implement `extract_frames(existing_buffer, received_chunk)`.
- Keep the function concerned with bytes; decode in the protocol or server layer.
- Write the deterministic tests from the previous page.
- Run `python -m unittest -v test_framing.py`.

Part D - Integrate only after unit tests pass

- Create one buffer inside each `handle_client()` call.
- Feed every positive `recv` chunk to the framing function.
- Decode and temporarily display every returned complete frame.
- Apply the EOF and maximum-size policies.

11. Assessment and review

Explain without looking

1. Why can a six-byte message require several `recv(1024)` calls?
2. Why can two `sendall()` calls appear in one `recv` chunk?
3. What is the difference between chunk size and maximum frame size?
4. Why does every worker need a separate buffer?
5. Why should UTF-8 decoding happen after framing?
6. What does `b''` from `recv()` mean?
7. How does a size limit prevent a memory-exhaustion bug?
8. Why does transient buffering still fit an ephemeral-chat requirement?

Practical evidence

- All deterministic framing tests pass.
- One frame split over multiple inputs is emitted once and only when complete.
- Several frames in one input are all emitted immediately and in order.
- Unicode text survives arbitrary byte chunking.
- Oversized unfinished input is rejected without unbounded growth.
- Three connected clients keep independent remainder buffers.

Official reading

[Python Socket Programming HOWTO](#) - read the sections on `send`, `recv`, and message framing.

[Python socket reference](#) - use for exact operation contracts.

[Python asyncio streams](#) - compare `readuntil`, `readexactly`, and configured limits.

[RFC 9293](#) - the formal TCP byte-stream specification.

Next milestone: complete the framing module before adding message broadcasting or disappearing-message timers.